

SQL Practice — Answer Key

Topic 1: DDL

1.

```
CREATE TABLE students (  
    student_id INTEGER PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    age INTEGER CHECK (age > 6 AND age < 25)  
);
```

2.

```
CREATE TABLE users (  
    user_id INTEGER PRIMARY KEY AUTO_INCREMENT,  
    email VARCHAR(150) NOT NULL,  
    CONSTRAINT users_email_unique UNIQUE (email)  
);
```

3.

```
ALTER TABLE customers ADD COLUMN password VARCHAR(255) NOT NULL;
```

4.

```
ALTER TABLE customers ADD COLUMN surname VARCHAR(100) AFTER name;
```

5.

```
ALTER TABLE customers DROP COLUMN surname;
```

6.

```
ALTER TABLE customers MODIFY COLUMN age INTEGER NOT NULL;
```

7.

```
ALTER TABLE customers ADD CONSTRAINT chk_age CHECK (age > 13);
```

8.

```
CREATE TABLE orders (  
    order_id INTEGER PRIMARY KEY AUTO_INCREMENT,  
    cid INTEGER NOT NULL,  
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    amount DECIMAL(10,2),  
    CONSTRAINT orders_fk FOREIGN KEY (cid) REFERENCES customers(cid) ON DELETE CASCADE  
);
```

9.

```
ALTER TABLE orders RENAME TO customer_orders;
```

10.

```
ALTER TABLE orders DROP FOREIGN KEY orders_fk;
```

Topic 2: DML

11.

```
INSERT INTO customers (name, email, age, city) VALUES
('Amit Sharma', 'amit@mail.com', 28, 'Delhi'),
('Priya Singh', 'priya@mail.com', 32, 'Mumbai'),
('Rahul Verma', 'rahul@mail.com', 25, 'Pune');
```

12.

```
INSERT INTO smartphones (brand, model, price, rating)
VALUES ('Samsung', 'Galaxy S24', 79999, 4.5);
```

13.

```
UPDATE smartphones
SET price = price * 0.90
WHERE brand = 'Samsung';
```

14.

```
DELETE FROM customers WHERE age > 60;
```

15.

```
DELETE FROM smartphones WHERE price > 200000;
```

16.

```
UPDATE customers SET city = 'Mumbai' WHERE cid = 5;
```

Topic 3: Aggregate & Scalar Functions

17.

```
SELECT MIN(price), MAX(price) FROM smartphones;
```

18.

```
SELECT MAX(price) FROM smartphones WHERE brand = 'Samsung';
```

19.

```
SELECT AVG(rating) FROM smartphones WHERE brand = 'Apple';
```

20.

```
SELECT COUNT(*) FROM smartphones WHERE brand = 'OnePlus';
```

21.

```
SELECT COUNT(DISTINCT brand) FROM smartphones;
```

22.

```
SELECT STD(screen_size) FROM smartphones;
```

23.

```
SELECT VARIANCE(price) FROM smartphones WHERE brand = 'Xiaomi';
```

24.

```
SELECT model, rating,  
       ABS(rating - (SELECT AVG(rating) FROM smartphones WHERE brand = 'Samsung')) AS diff_from_avg  
FROM smartphones  
WHERE brand = 'Samsung';
```

25.

```
SELECT ROUND(ppi, 1) FROM smartphones;
```

26.

```
SELECT FLOOR(rating), CEIL(rating) FROM smartphones;
```

27.

```
SELECT AVG(battery_capacity), AVG(primary_camera_rear)  
FROM smartphones  
WHERE price >= 100000;
```

28.

```
SELECT AVG(internal_memory)  
FROM smartphones  
WHERE refresh_rate >= 120 AND primary_camera_front >= 20;
```

29.

```
SELECT COUNT(*) FROM smartphones WHERE has_5g = TRUE;
```

Topic 4: Sorting

30.

```
SELECT * FROM smartphones  
WHERE brand = 'Samsung'  
ORDER BY screen_size DESC  
LIMIT 5;
```

31.

```
SELECT * FROM smartphones  
ORDER BY (primary_camera_rear + primary_camera_front) DESC;
```

32.

```
SELECT * FROM smartphones ORDER BY ppi DESC;
```

33.

```
SELECT * FROM smartphones  
ORDER BY battery_capacity DESC  
LIMIT 1 OFFSET 1;
```

34.

```
SELECT model, rating FROM smartphones
WHERE brand = 'Apple'
ORDER BY rating ASC
LIMIT 1;
```

35.

```
SELECT * FROM smartphones
ORDER BY brand ASC, rating DESC;
```

36.

```
SELECT * FROM smartphones
ORDER BY brand ASC, price ASC;
```

Topic 5: Grouping & HAVING

37.

```
SELECT brand,
       COUNT(*) AS total_phones,
       AVG(price) AS avg_price,
       MAX(rating) AS max_rating,
       AVG(screen_size) AS avg_screen_size,
       AVG(battery_capacity) AS avg_battery
FROM smartphones
GROUP BY brand;
```

38.

```
SELECT has_nfc, AVG(price), AVG(rating)
FROM smartphones
GROUP BY has_nfc;
```

39.

```
SELECT brand, processor_brand,
       COUNT(*) AS model_count,
       AVG(primary_camera_rear) AS avg_rear_camera
FROM smartphones
GROUP BY brand, processor_brand;
```

40.

```
SELECT brand, AVG(price) AS avg_price
FROM smartphones
GROUP BY brand
ORDER BY avg_price DESC
LIMIT 5;
```

41.

```
SELECT brand, MIN(screen_size) AS min_screen
FROM smartphones
GROUP BY brand
```

```
ORDER BY min_screen ASC
LIMIT 1;
```

42.

```
SELECT has_5g, AVG(price)
FROM smartphones
GROUP BY has_5g;
```

43.

```
SELECT brand, COUNT(*) AS model_count
FROM smartphones
WHERE has_nfc = TRUE AND has_5g = TRUE
GROUP BY brand
ORDER BY model_count DESC
LIMIT 1;
```

44.

```
SELECT has_nfc, AVG(price)
FROM smartphones
WHERE brand = 'Samsung' AND has_5g = TRUE
GROUP BY has_nfc;
```

45.

```
SELECT model, price FROM smartphones
ORDER BY price DESC
LIMIT 1;
```

46.

```
SELECT brand, AVG(rating) AS avg_rating
FROM smartphones
GROUP BY brand
HAVING COUNT(*) > 20;
```

47.

```
SELECT brand, AVG(ram) AS avg_ram
FROM smartphones
WHERE refresh_rate >= 90 AND has_fast_charging = TRUE
GROUP BY brand
HAVING COUNT(*) >= 10
ORDER BY avg_ram DESC
LIMIT 3;
```

48.

```
SELECT brand, AVG(price) AS avg_price
FROM smartphones
WHERE has_5g = TRUE
GROUP BY brand
HAVING AVG(rating) > 70 AND COUNT(*) > 10;
```

Topic 6: Joins

49.

```
SELECT o.order_id, c.name
FROM orders o
INNER JOIN customers c ON o.cid = c.cid;
```

50.

```
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.cid = o.cid;
```

51.

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.cid = o.cid
WHERE o.order_id IS NULL;
```

52.

```
SELECT c.name, o.order_id
FROM customers c
RIGHT JOIN orders o ON c.cid = o.cid;
```

53.

```
SELECT c.name, o.order_id FROM customers c
LEFT JOIN orders o ON c.cid = o.cid
UNION
SELECT c.name, o.order_id FROM customers c
RIGHT JOIN orders o ON c.cid = o.cid;
```

54.

```
SELECT e1.name AS employee, e2.name AS manager
FROM employees e1
JOIN employees e2 ON e1.manager_id = e2.employee_id;
```

55.

```
-- Example: every size combined with every color
SELECT * FROM sizes
CROSS JOIN colors;
-- Returns rows(sizes) x rows(colors) total combinations, no condition needed.
```

56.

```
SELECT *
FROM order_items oi
JOIN products p
  ON oi.product_id = p.product_id
  AND oi.variant_id = p.variant_id;
```

57.

```
SELECT o.order_id, c.name, oi.item_name
FROM orders o
```

```
JOIN customers c ON o.cid = c.cid
JOIN order_items oi ON o.order_id = oi.order_id;
```

Topic 7: Set Operations

58.

```
SELECT name FROM customers WHERE city = 'Delhi'
UNION
SELECT name FROM customers WHERE city = 'Mumbai';
```

59.

```
-- UNION removes duplicates
SELECT city FROM customers
UNION
SELECT city FROM customers; -- duplicates removed

-- UNION ALL keeps duplicates (faster, no dedup step)
SELECT city FROM customers
UNION ALL
SELECT city FROM customers; -- all rows kept, including duplicates
```

60.

```
-- Simulating INTERSECT: customers present in both Delhi orders and Mumbai orders (example logic)
SELECT a.name
FROM (SELECT name FROM customers WHERE city = 'Delhi') a
INNER JOIN (SELECT name FROM customers WHERE age > 25) b
ON a.name = b.name;
```

61.

```
-- Simulating EXCEPT: customers in Delhi but NOT in Mumbai
SELECT name FROM customers WHERE city = 'Delhi'
AND name NOT IN (SELECT name FROM customers WHERE city = 'Mumbai');
```

Topic 8: Query Execution Order

62.

> Because SQL executes `WHERE` **before** `GROUP BY`/`SELECT`. At the `WHERE` stage, rows haven't been grouped yet, so aggregate values like `AVG()` or `COUNT()` don't exist. Aggregate functions are only valid after grouping — hence they belong in `HAVING`, which runs after `GROUP BY`.

63.

```
SELECT brand, AVG(price) AS avg_price, COUNT(*) AS total
FROM smartphones
WHERE has_5g = TRUE
GROUP BY brand
HAVING COUNT(*) > 5
ORDER BY avg_price DESC
LIMIT 3;
```

Topic 9: Mixed / Practice (IPL-style)

64.

```
SELECT batsman, SUM(runs) AS total_runs
FROM ipl_delivery
GROUP BY batsman
ORDER BY total_runs DESC
LIMIT 5;
```

65.

```
SELECT batsman, COUNT(*) AS sixes
FROM ipl_delivery
WHERE runs = 6
GROUP BY batsman
ORDER BY sixes DESC
LIMIT 1 OFFSET 1;
```

66.

```
SELECT bowling_team, SUM(runs) AS total_runs
FROM ipl_delivery
WHERE batsman = 'V Kohli'
GROUP BY bowling_team
ORDER BY total_runs DESC;
```

67.

```
SELECT batsman, match_id, SUM(runs) AS match_runs
FROM ipl_delivery
GROUP BY batsman, match_id
HAVING SUM(runs) >= 100
ORDER BY match_runs DESC
LIMIT 10;
```

68.

```
SELECT batsman,
       SUM(runs) AS total_runs,
       COUNT(*) AS balls_faced,
       (SUM(runs) / COUNT(*)) * 100 AS strike_rate
FROM ipl_delivery
GROUP BY batsman
HAVING balls_faced >= 1000
ORDER BY strike_rate DESC
LIMIT 5;
```